FP065 · Desarrollo back-end

Producto 2

Aplicación web de gestión de Biblioteca

Spring Boot + Spring Data JPA + Thymeleaf + MariaDB + Docker

Alumna: Andrea Valero Sanz

Módulo: FP065 · Desarrollo back-end

Entrega: Producto 2

Curso: 2026/2027

Índice

1. Introducción.....4
1.1. Tecnologías utilizadas.....4
2. Requisitos.....4
2.1. Requisitos funcionales.....4
2.2. Requisitos no funcionales.....5
3. Diseño de la base de datos.....5
3.1. Entidades y relaciones.....5
3.2. Diagrama Entidad/Relación.....6
3.3. Modelo relacional.....7
4. Mockups de la interfaz.....7
5. Arquitectura de la aplicación (MVC).....11
5.1. Modelo (entidades JPA).....11
5.2. Repositorios (Spring Data JPA).....12
5.3. Controladores.....12
5.4. Servicios (lógica de negocio).....13
5.5. Vistas (Thymeleaf + Bootstrap).....13
6. Configuración de la base de datos.....15
6.1. application.properties.....15
6.2. Script de inicialización db/init.sql.....15
7. Docker.....16
7.1. Puesta en marcha.....17
8. Trabajo realizado.....17
8.1. Estado final por entidad.....18
8.2. Pruebas automatizadas.....18
8.3. Capturas de la aplicación en funcionamiento.....18
8.4. Vídeo demostrativo.....24
9. Dificultades encontradas.....24
10. Puntos no realizados.....24

11. Propuestas de mejora.....25
12. Autoevaluación.....25
13. Webgrafía.....26

1. Introducción

Este documento es la memoria del **Producto 2** del módulo FP065. El objetivo es desarrollar una aplicación web del lado servidor que gestione los datos de una **biblioteca**: su catálogo de libros, los autores de cada libro, los usuarios (socios) y los préstamos de libros a esos usuarios.

La aplicación sigue el patrón **Modelo-Vista-Controlador (MVC)** con **Spring Boot**. La persistencia se implementa con **Spring Data JPA** (Hibernate) sobre una base de datos **MariaDB**, las vistas se generan en el servidor con **Thymeleaf** y se maquetan con **Bootstrap 5**. Toda la solución (aplicación + base de datos) se levanta con **Docker Compose**, de forma que cualquiera puede ejecutarla con un único comando sin instalar Java ni MariaDB.

El código fuente está en el repositorio (privado) **https://github.com/andreavalerosanz-rgb/biblioteca-p2**.

La memoria recoge los requisitos, el diseño de la base de datos (modelo E/R y modelo relacional), los mockups de las pantallas, la arquitectura del código, la configuración de la base de datos y de Docker, el trabajo realizado, las dificultades encontradas, las mejoras propuestas y una autoevaluación según la rúbrica.

1.1. Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Lenguaje	Java 25	Código de la aplicación
Framework	Spring Boot 4 (spring-boot-starter-webmvc)	Arranque, servidor embebido, controladores MVC
Persistencia	Spring Data JPA + Hibernate	Entidades, repositorios y consultas derivadas
Validación	Jakarta Bean Validation	@NotBlank, @Email, @Min… en las entidades
Vistas	Thymeleaf + Bootstrap 5 + Bootstrap Icons	Plantillas HTML renderizadas en servidor y hoja de estilos propia (biblioteca.css)
Base de datos	MariaDB 11	Almacenamiento; driver mariadb-java-client
Contenedores	Docker + Docker Compose	Imagen multi-etapa de la app y servicio de BD
Construcción	Maven (wrapper mvnw)	Compilación y empaquetado en JAR

2. Requisitos

2.1. Requisitos funcionales

ID	Requisito
RF-01	Mostrar una página de inicio con el número de libros, usuarios y préstamos activos, y los

	últimos préstamos realizados.
RF-02	Listar, crear, editar y eliminar **libros** (título, ISBN, editorial, año de publicación y ejemplares disponibles).
RF-03	Asignar a cada libro **uno o varios autores** mediante casillas de selección múltiple (relación N:M).
RF-04	Listar y dar de alta **autores** (nombre y nacionalidad), mostrando cuántos libros tiene cada uno.
RF-05	Listar, crear, editar y eliminar **usuarios** (nombre, apellidos, email único, teléfono y fecha de alta).
RF-06	Registrar un **préstamo** eligiendo usuario y libro, con fecha de préstamo y fecha de devolución prevista.
RF-07	**Devolver** un préstamo: se guarda la fecha de devolución, el estado pasa a DEVUELTO y se repone el ejemplar.
RF-08	Mostrar el **estado** de cada préstamo (ACTIVO, DEVUELTO, RETRASADO) con un distintivo de color.
RF-09	No permitir prestar un libro sin ejemplares disponibles ni borrar libros/usuarios con préstamos asociados.
RF-10	Validar los formularios y mostrar mensajes de error junto al campo y avisos de éxito tras cada operación.

2.2. Requisitos no funcionales

- **Arquitectura MVC** con separación en paquetes `model`, `repository`, `controller` y plantillas en `templates`.
- **Persistencia en MariaDB** con integridad referencial (claves foráneas) y restricciones de unicidad (email, ISBN).
- **Despliegue reproducible con Docker Compose**: la base de datos se inicializa sola con `db/init.sql`.
- **Configuración externalizada**: credenciales y URL de la BD por variables de entorno con valores por defecto.
- Interfaz **responsive** y homogénea con Bootstrap 5: barra de navegación común y estilos de botón coherentes.
- Textos de la interfaz y del código de dominio **en español**; codificación UTF-8 (utf8mb4) de extremo a extremo.

3. Diseño de la base de datos

3.1. Entidades y relaciones

El dominio se modela con cuatro entidades:

- **Usuario**: socio de la biblioteca. Atributos: id, nombre, apellidos, email (único), teléfono, fecha de alta.

- **Libro**: obra del catálogo. Atributos: id, título, ISBN (único), editorial, año de publicación, ejemplares disponibles.
- **Autor**: escritor. Atributos: id, nombre, nacionalidad.
- **Préstamo**: un libro prestado a un usuario. Atributos: id, fecha de préstamo, fecha de devolución prevista, fecha de devolución (vacía mientras no se devuelve) y estado.

Relaciones:

Relación	Cardinalidad	Significado
Usuario – realiza – Préstamo	1 : N	Un usuario puede tener muchos préstamos; cada préstamo es de un único usuario.
Libro – se presta en – Préstamo	1 : N	Un libro puede prestarse muchas veces; cada préstamo es de un único libro.
Libro – escrito por – Autor	**N : M**	Un libro puede tener varios autores y un autor puede haber escrito varios libros.

La relación **N:M entre Libro y Autor** (exigida por la rúbrica) se transforma en el modelo relacional en la tabla intermedia `libro_autor`, cuya clave primaria es compuesta (`libro_id`, `autor_id`) y cuyas dos columnas son claves foráneas. «Préstamo» se modela como entidad propia (y no como atributo de una relación N:M Usuario–Libro) porque tiene identidad, estado e historial: el mismo usuario puede tomar el mismo libro varias veces.

3.2. Diagrama Entidad/Relación

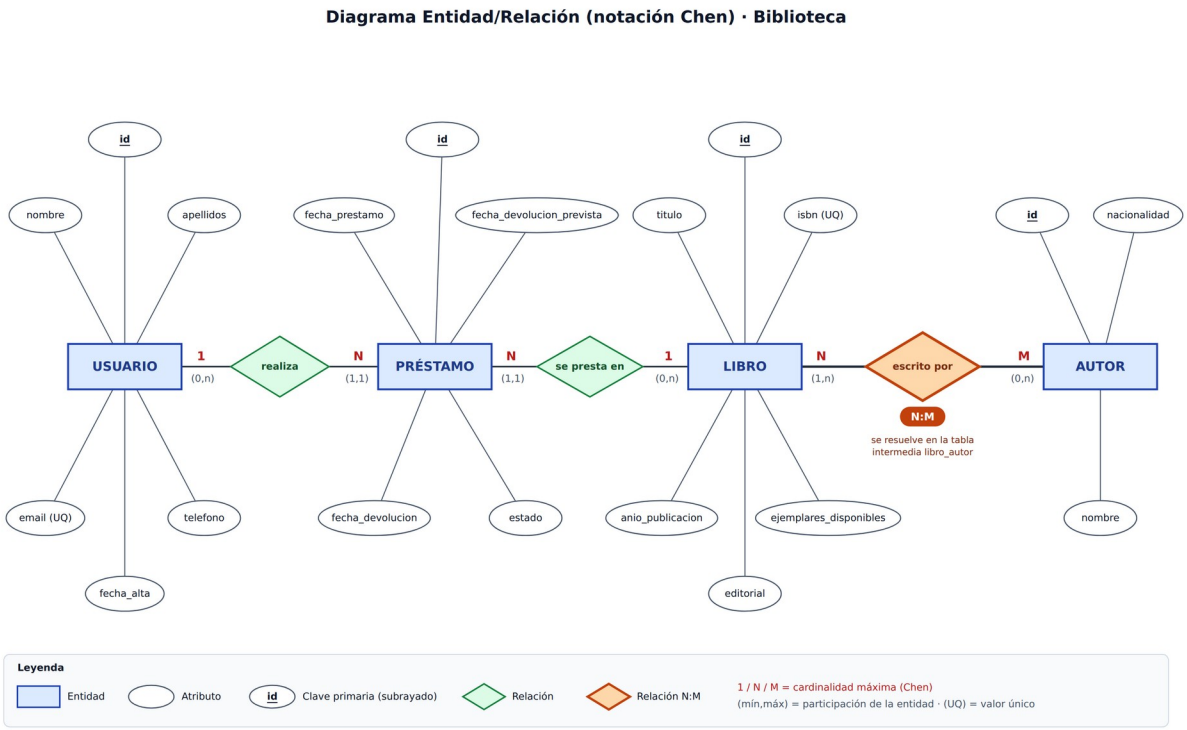


Figura 1. Diagrama Entidad/Relación (notación Chen)

3.3. Modelo relacional

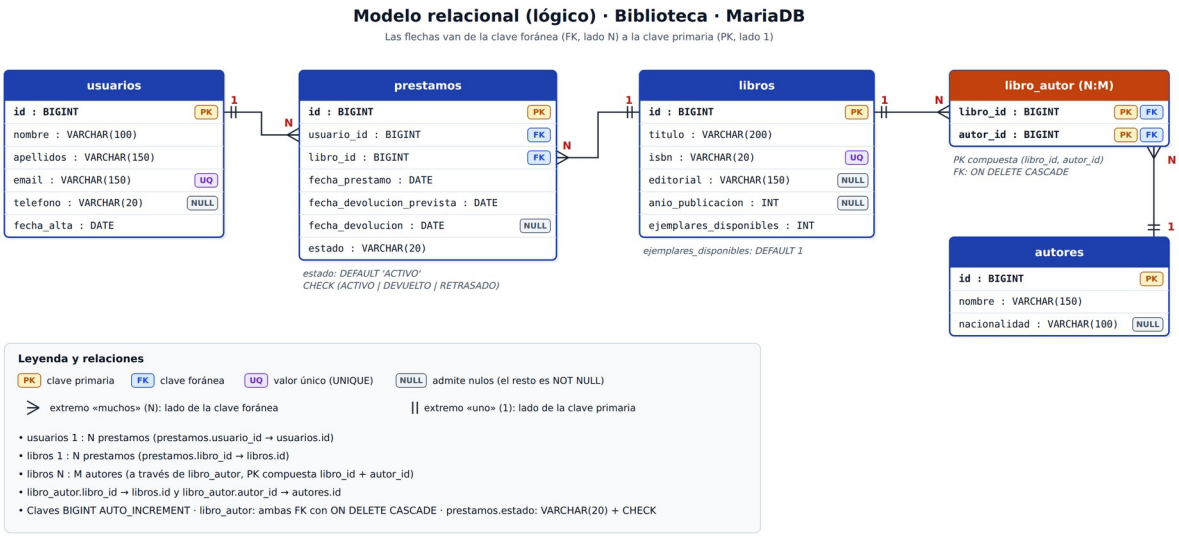


Figura 2. Modelo relacional (tablas, claves primarias y foráneas)

Tabla	Clave primaria	Claves foráneas	Restricciones
usuarios	id	—	email UNIQUE
libros	id	—	isbn UNIQUE; ejemplares_disponibles ≥ 0
autores	id	—	—
libro_autor	(libro_id, autor_id)	libro_id → libros.id; autor_id → autores.id	ON DELETE CASCADE
prestamos	id	usuario_id → usuarios.id; libro_id → libros.id	estado IN (ACTIVO, DEVUELTO, RETRASADO); fecha_devolucion NULL

Los archivos fuente editables de los diagramas están en `docs/diagramas/` (script generador, Mermaid y draw.io).

4. Mockups de la interfaz

Antes de programar las vistas se diseñaron los mockups (wireframes) de las pantallas principales. Todas comparten una **barra de navegación oscura** con la marca «Biblioteca» y los enlaces Inicio, Libros, Usuarios, Préstamos y Autores. Los listados tienen el botón azul «+ Nuevo» arriba a la derecha y, en cada fila, los botones **Editar** (amarillo) y **Eliminar** (rojo); los préstamos pendientes muestran el botón **Devolver** (verde). Los HTML de los mockups están en `docs/mockups/`.

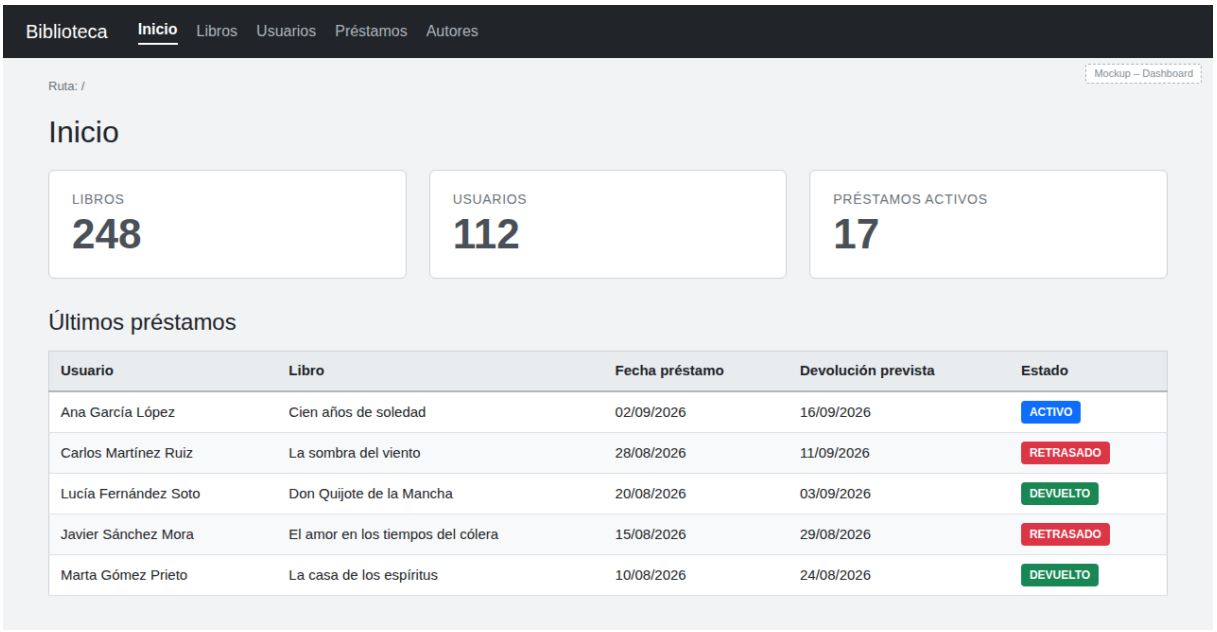


Figura 3. Mockup – Inicio / dashboard (ruta /)

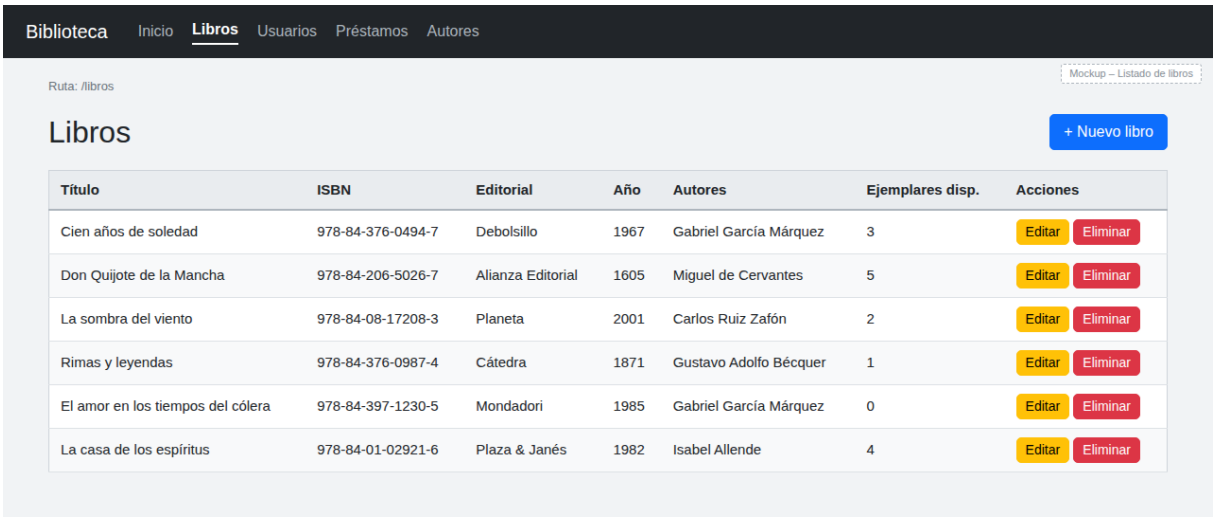


Figura 4. Mockup – Listado de libros (/libros)

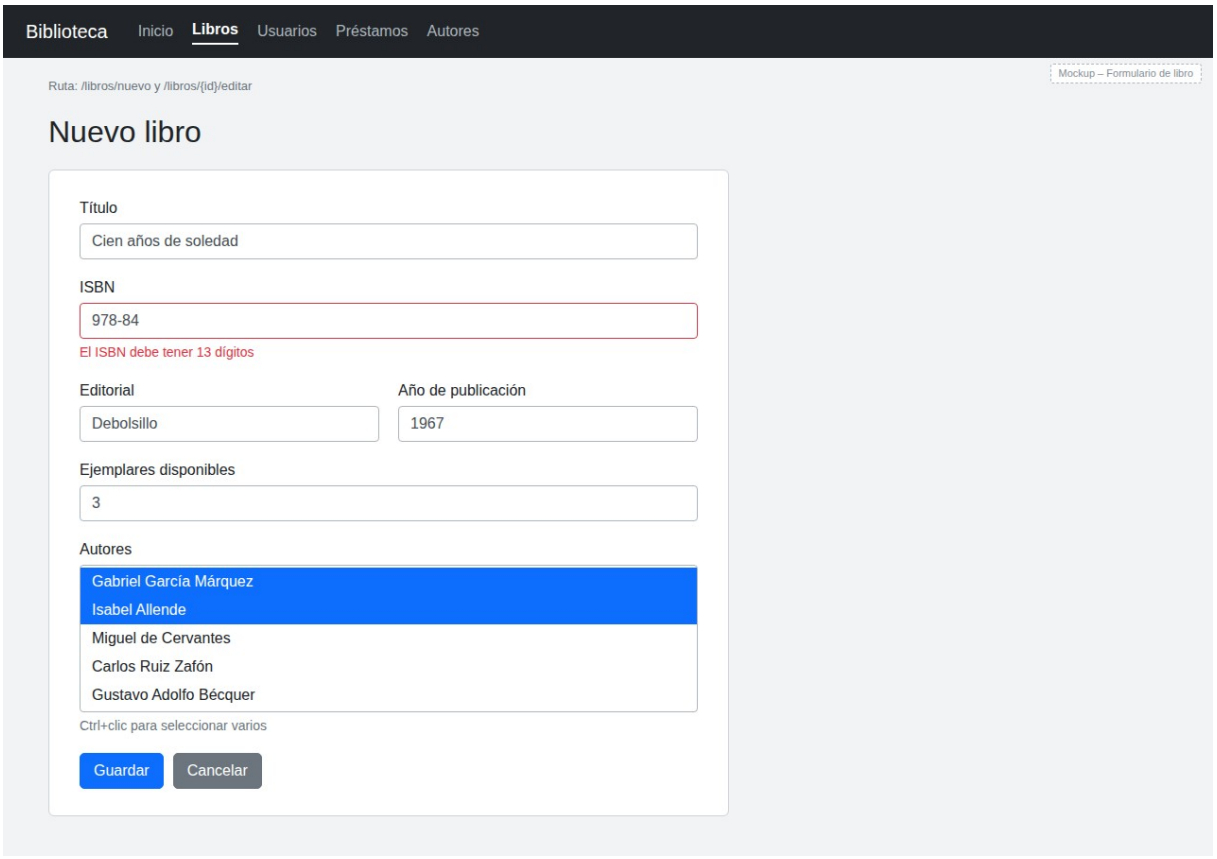


Figura 5. Mockup – Formulario de libro (/libros/nuevo, /libros/{id}/editar)

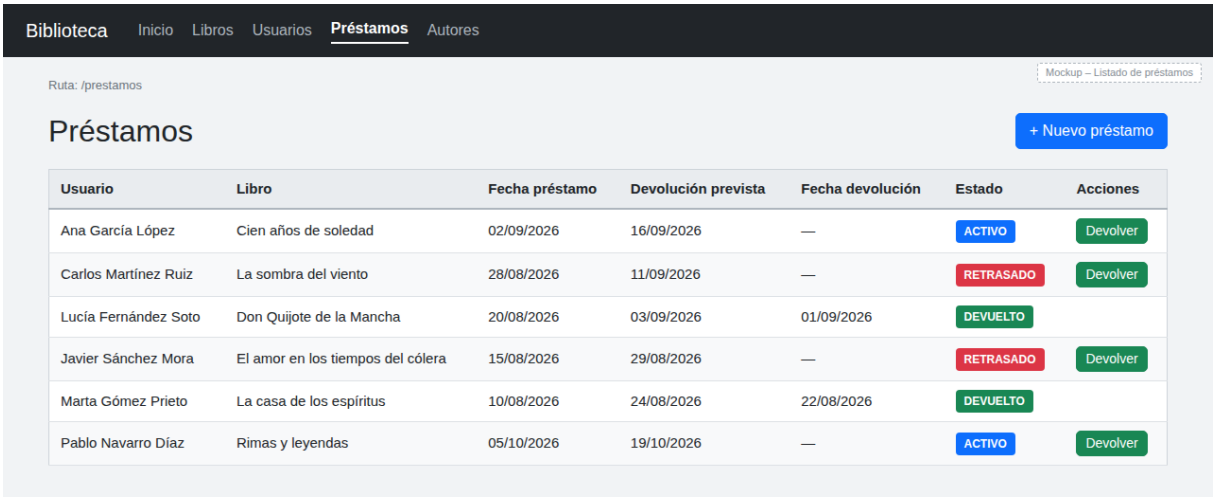


Figura 6. Mockup – Listado de préstamos (/prestamos)

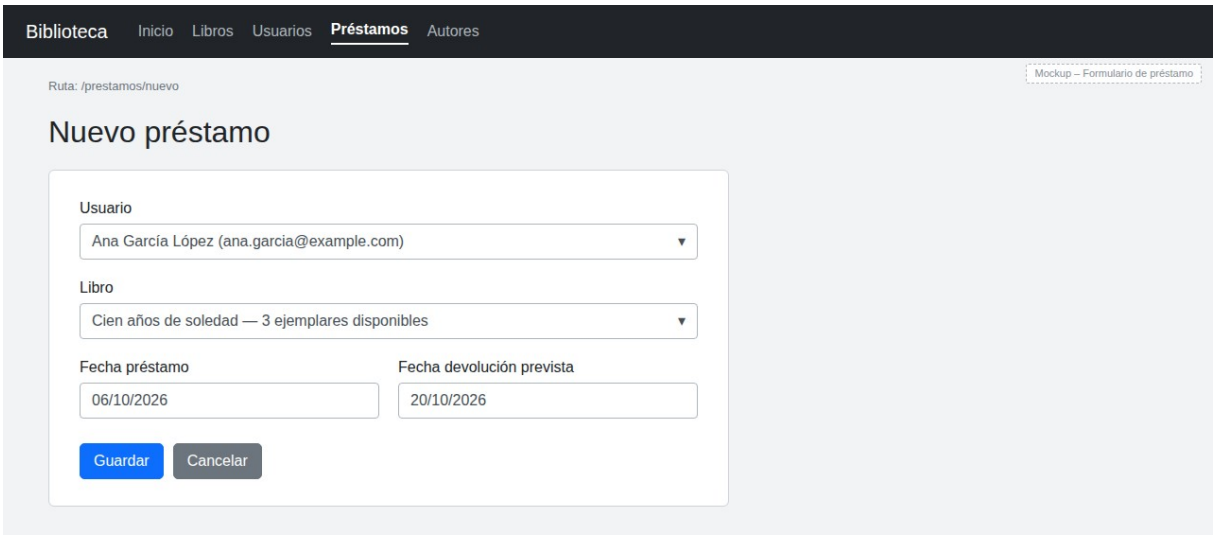


Figura 7. Mockup – Nuevo préstamo (/prestamos/nuevo)

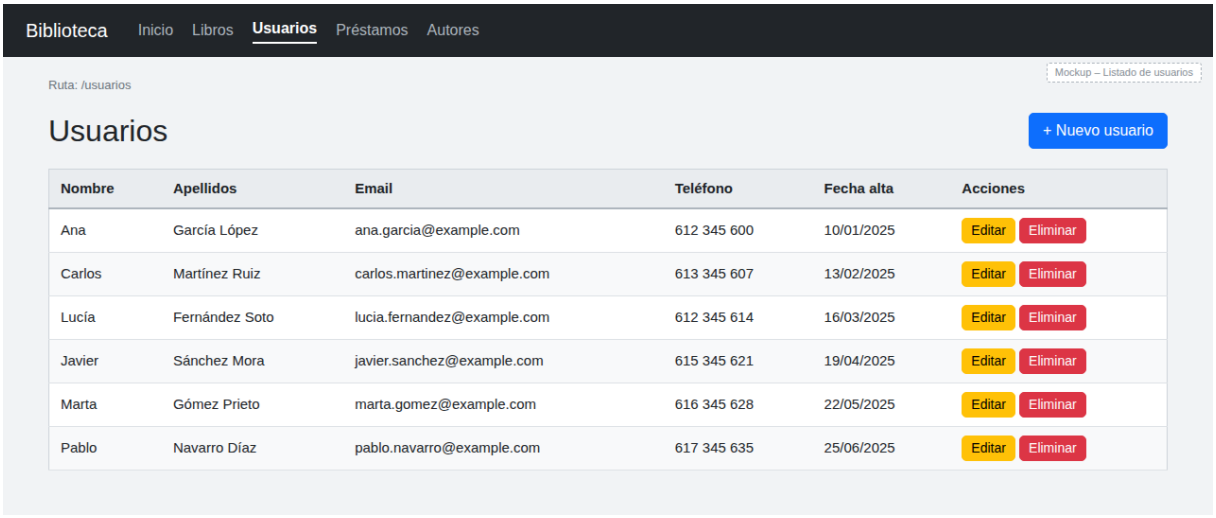


Figura 8. Mockup – Listado de usuarios (/usuarios)

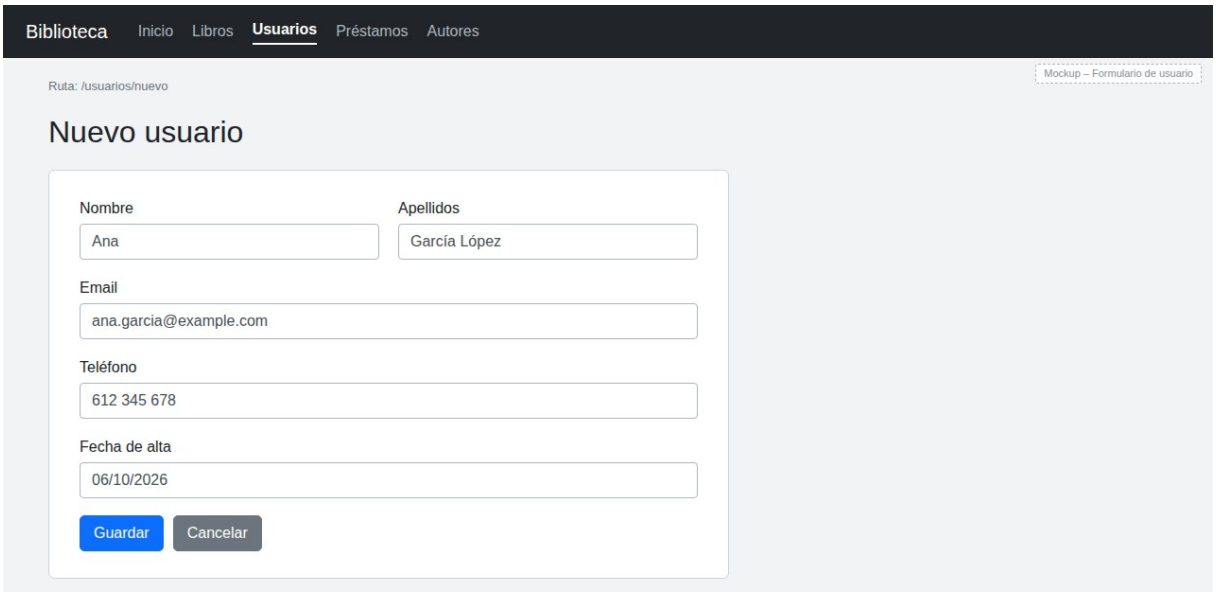


Figura 9. Mockup – Formulario de usuario (/usuarios/nuevo)

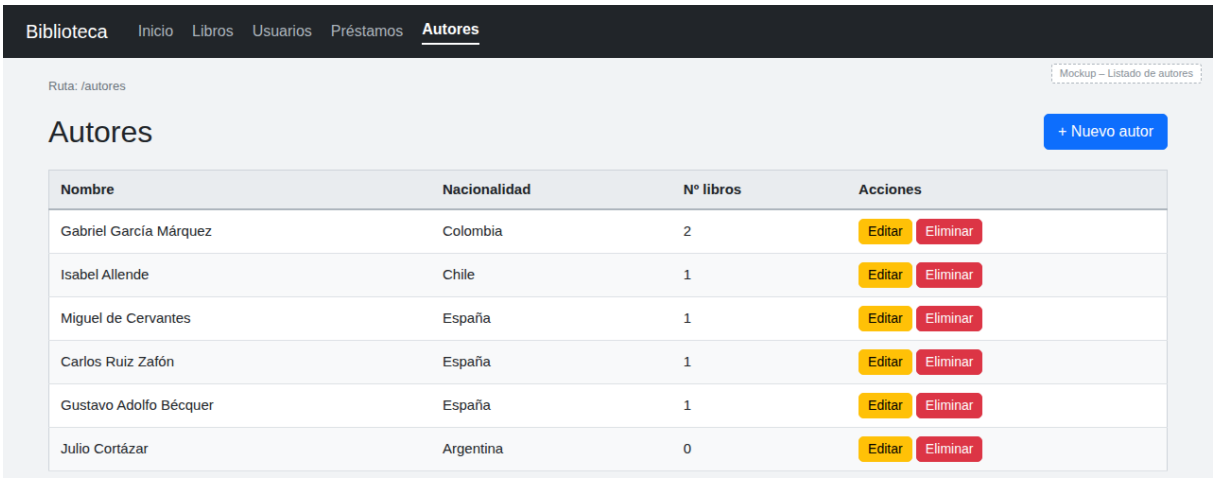


Figura 10. Mockup – Listado de autores (/autores)

5. Arquitectura de la aplicación (MVC)

El código se organiza en el paquete `com.serverside.appbackendp1` siguiendo el patrón MVC:

Estructura de paquetes

```
src/main/java/com/serverside/appbackendp1/
├── AppBackendP1Application.java   ← clase principal (@SpringBootApplication)
├── model/          ← Modelo: entidades JPA
│   ├── Usuario.java, Libro.java, Autor.java, Prestamo.java
│   └── EstadoPrestamo.java (enum ACTIVO | DEVUELTO | RETRASADO)
├── repository/     ← Acceso a datos: interfaces JpaRepository
│   └── UsuarioRepository, LibroRepository, AutorRepository, PrestamoRepository
├── service/        ← Lógica de negocio (@Service, @Transactional)
│   └── LibroService, UsuarioService, PrestamoService
└── controller/     ← Controlador: @Controller por entidad + HomeController
src/main/resources/
├── application.properties
└── templates/      ← Vista: plantillas Thymeleaf
    ├── fragments/layout.html (cabecera, navbar, mensajes)
    ├── index.html, libros/, usuarios/, prestamos/, autores/
```

5.1. Modelo (entidades JPA)

Cada tabla se representa con una clase anotada con `@Entity` y `@Table`. Las claves primarias usan `@GeneratedValue(strategy = GenerationType.IDENTITY)` (AUTO_INCREMENT en MariaDB). Las restricciones de validación (`@NotBlank`, `@Email`, `@Min`, `@NotNull`) se declaran en los propios atributos y Spring las comprueba al recibir el formulario con `@Valid`.

Libro.java – relación N:M con Autor y 1:N con Préstamo

```
    @ManyToMany
    @JoinTable(name = "libro_autor",
            joinColumns = @JoinColumn(name = "libro_id"),
            inverseJoinColumns = @JoinColumn(name = "autor_id"))
    private Set<Autor> autores = new HashSet<>();

    @OneToMany(mappedBy = "libro")
    private List<Prestamo> prestamos = new ArrayList<>();
```

Prestamo.java – relaciones N:1 con Usuario y Libro

```
    @ManyToOne(optional = false)
    @JoinColumn(name = "usuario_id", nullable = false)
    private Usuario usuario;

    @NotNull(message = "Selecciona un libro")
    @ManyToOne(optional = false)
    @JoinColumn(name = "libro_id", nullable = false)
    private Libro libro;
```

La relación N:M se mapea con `@ManyToMany` + `@JoinTable(name = "libro_autor")` en el lado propietario (Libro) y `@ManyToMany(mappedBy = "autores")` en Autor. El estado del préstamo es un `enum` guardado como texto (`@Enumerated(EnumType.STRING)`).

5.2. Repositorios (Spring Data JPA)

Los repositorios son interfaces que extienden `JpaRepository<Entidad, Long>`; Spring genera la implementación en tiempo de ejecución (CRUD completo: `findAll`, `findById`, `save`, `deleteById`…). Además se definen **consultas derivadas** a partir del nombre del método:

PrestamoRepository.java

```
public interface PrestamoRepository extends JpaRepository<Prestamo, Long> {

    List<Prestamo> findAllByOrderByFechaPrestamoDescIdDesc();

    List<Prestamo> findTop5ByOrderByFechaPrestamoDescIdDesc();

    long countByEstado(EstadoPrestamo estado);

    boolean existsByLibroId(Long libroId);

    boolean existsByUsuarioId(Long usuarioId);
}
```

5.3. Controladores

Cada entidad tiene un `@Controller` que atiende las rutas de su listado y formularios. Los métodos devuelven el nombre de la plantilla Thymeleaf y pasan los datos a través del `Model`. Tras guardar o eliminar se aplica el patrón **Post/Redirect/Get** (`redirect:/libros`) con un mensaje flash (`RedirectAttributes`).

Método y ruta	Acción
GET /	Inicio: contadores y últimos 5 préstamos
GET /libros	Listado de libros
GET/POST /libros/nuevo	Formulario de alta y guardado (con @Valid)
GET/POST /libros/{id}/editar	Formulario de edición y guardado
POST /libros/{id}/eliminar	Baja (se impide si tiene préstamos)
/usuarios, /usuarios/nuevo, /usuarios/{id}/editar, /usuarios/{id}/eliminar	Gestión de usuarios (mismo esquema)
GET /autores · GET/POST /autores/nuevo	Listado y alta de autores

GET /prestamos · GET /prestamos/nuevo · POST /prestamos	Listado y alta de préstamos (descuenta un ejemplar)
POST /prestamos/{id}/devolver	Marca DEVUELTO, guarda la fecha y repone el ejemplar

PrestamoController.java – acción Devolver

```
    @PostMapping("/{id}/devolver")
    public String devolver(@PathVariable Long id, RedirectAttributes ra) {
        if (!prestamoService.existe(id)) {
            ra.addFlashAttribute("error", "El préstamo solicitado no existe");
        } else if (prestamoService.devolver(id)) {
            ra.addFlashAttribute("exito", "Préstamo marcado como devuelto");
        } else {
            ra.addFlashAttribute("error", "El préstamo ya estaba devuelto");
        }
        return "redirect:/prestamos";
    }
```

5.4. Servicios (lógica de negocio)

Entre controladores y repositorios hay una capa de servicios (`@Service`). Los métodos que modifican varios datos a la vez se marcan con `@Transactional`, de modo que crear un préstamo (insertar el préstamo y descontar un ejemplar) o devolverlo (cambiar el estado, guardar la fecha y reponer el ejemplar) se ejecuta de forma atómica. `PrestamoService` también recalcula el estado **RETRASADO** de los préstamos activos vencidos cada vez que se consultan los listados.

PrestamoService.java – devolución de un préstamo

```
    /** Devuelve true si se marcó como devuelto; false si no existe o ya estaba
devuelto. */
    @Transactional
    public boolean devolver(Long id) {
        Prestamo p = prestamoRepository.findById(id).orElse(null);
        if (p == null || p.getEstado() == EstadoPrestamo.DEVUELTO) {
            return false;
        }
        p.setEstado(EstadoPrestamo.DEVUELTO);
        p.setFechaDevolucion(LocalDate.now());
        Libro libro = p.getLibro();
        libro.setEjemplaresDisponibles(libro.getEjemplaresDisponibles() + 1);
        return true;
    }

    @Transactional(readOnly = true)
    public boolean existe(Long id) {
```

5.5. Vistas (Thymeleaf + Bootstrap)

Las plantillas están en `src/main/resources/templates`. Los elementos comunes (cabecera `<head>`, barra de navegación y mensajes flash) se definen una sola vez en `fragments/layout.html` y se reutilizan con `th:replace`. Los formularios se enlazan al objeto del modelo con `th:object` y `th:field`, y los errores de validación se muestran con `th:errors`.

El aspecto final se diseñó sobre los mockups con la idea de la **ficha catalográfica** del fichero de una biblioteca: cada listado y formulario es una «ficha» blanca con el filete rojo doble bajo la cabecera y el margen rojo vertical; los datos (signaturas, ISBN, fechas) van en tipografía mecanográfica (Courier Prime) y la interfaz en Atkinson Hyperlegible, pensada para la legibilidad. Los estados de los préstamos se muestran como sellos de tinta y la fila de un préstamo retrasado se tiñe de rojo. Cada libro muestra su **signatura topográfica** (tres letras del primer apellido del autor y tres del título, como en el tejuelo), calculada en la propia plantilla. Los estilos están en `static/css/biblioteca.css`, sobre Bootstrap 5, y cumplen WCAG 2.1 AA (contraste, foco visible, etiquetas en todos los campos, enlace «Saltar al contenido») y se adaptan a móvil.

fragments/layout.html – fragmento de la signatura topográfica

```
<!-- Signatura topográfica: tres letras del primer apellido del autor y tres del título
sin artículo -->
<span th:fragment="signatura(l)" class="signatura"
      th:with="nombres=${#lists.sort(l.autores.![nombre])},
               apellido=${nombres.isEmpty()} ? '' : ${nombres[0].replaceFirst('^[^ ]+ +
((de|del|la) +)*', '').toUpperCase()},
               titulo=${l.titulo == null} ? '' : ${l.titulo.replaceFirst('^(?i)(el|la|
los|las|un|una) +', '').toLowerCase()}">
    <span class="signatura-autor" th:text="${apellido.length() > 3} ? $
{apellido.substring(0, 3)} : (${apellido.isEmpty()} ? '—' : ${apellido})">GAR</span>
    <span class="signatura-titulo" th:text="${titulo.length() > 3} ? $
{titulo.substring(0, 3)} : ${titulo}">cie</span>
</span>
```

fragments/layout.html – barra de navegación común

```
<nav th:fragment="navbar(activo)" class="navbar navbar-expand-lg navbar-dark bg-dark
barra">
    <div class="container">
        <a class="navbar-brand" th:href="@{/}">
            <i class="bi bi-book" aria-hidden="true"></i>
            <span>Biblioteca</span>
        </a>
        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#menu"
                aria-controls="menu" aria-expanded="false" aria-label="Abrir menú">
            <span class="navbar-toggler-icon"></span>
        </button>
        <div class="collapse navbar-collapse" id="menu">
            <ul class="navbar-nav ms-auto">
                <li class="nav-item"><a class="nav-link" th:classappend="${activo ==
'inicio'} ? 'active'" th:attr="aria-current=${activo == 'inicio'} ? 'page'"
th:href="@{/}">Inicio</a></li>
                <li class="nav-item"><a class="nav-link" th:classappend="${activo ==
'libros'} ? 'active'" th:attr="aria-current=${activo == 'libros'} ? 'page'"
th:href="@{/libros}">Libros</a></li>
                <li class="nav-item"><a class="nav-link" th:classappend="${activo ==
'usuarios'} ? 'active'" th:attr="aria-current=${activo == 'usuarios'} ? 'page'"
th:href="@{/usuarios}">Usuarios</a></li>
                <li class="nav-item"><a class="nav-link" th:classappend="${activo ==
'prestamos'} ? 'active'" th:attr="aria-current=${activo == 'prestamos'} ? 'page'"
th:href="@{/prestamos}">Préstamos</a></li>
                <li class="nav-item"><a class="nav-link" th:classappend="${activo ==
'autores'} ? 'active'" th:attr="aria-current=${activo == 'autores'} ? 'page'"
th:href="@{/autores}">Autores</a></li>
            </ul>
        </div>
    </div>
</nav>
```

6. Configuración de la base de datos

6.1. application.properties

La conexión se define en `src/main/resources/application.properties`. Cada valor puede sobrescribirse con una variable de entorno (sintaxis `${VARIABLE:valor_por_defecto}`), lo que permite usar `localhost` en desarrollo y el nombre del servicio `db` dentro de Docker. Como el esquema lo crea `init.sql`, Hibernate se configura con `ddl-auto=validate`: solo comprueba que las entidades coinciden con las tablas.

application.properties

```
spring.application.name=app-backend-p1

# Base de datos MariaDB (sobrescribible con variables de entorno)
spring.datasource.url=${SPRING_DATASOURCE_URL:jdbc:mariadb://localhost:3306/biblioteca}
spring.datasource.username=${SPRING_DATASOURCE_USERNAME:biblioteca}
spring.datasource.password=${SPRING_DATASOURCE_PASSWORD:biblioteca}
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver

# El esquema lo crea db/init.sql; JPA solo valida que coincide con las entidades
spring.jpa.hibernate.ddl-auto=${SPRING_JPA_HIBERNATE_DDL_AUTO:validate}
spring.jpa.open-in-view=true
spring.jpa.show-sql=false

spring.thymeleaf.cache=false

# Sesión solo por cookie (evita ;jsessionid= en las URLs)
server.servlet.session.tracking-modes=cookie
```

6.2. Script de inicialización db/init.sql

El script `db/init.sql` crea la base de datos `biblioteca` (utf8mb4), las cinco tablas con sus claves primarias, foráneas, restricciones UNIQUE y CHECK, e inserta datos de ejemplo (usuarios, autores, libros, relaciones libro–autor y préstamos). MariaDB lo ejecuta automáticamente la primera vez que arranca el contenedor, al montarse en `/docker-entrypoint-initdb.d/`.

init.sql – tabla intermedia de la relación N:M

```
-- Relación N:M Libro <-> Autor
CREATE TABLE IF NOT EXISTS libro_autor (
    libro_id BIGINT NOT NULL,
    autor_id BIGINT NOT NULL,
    PRIMARY KEY (libro_id, autor_id),
    CONSTRAINT fk_libro_autor_libro FOREIGN KEY (libro_id) REFERENCES libros (id) ON
DELETE CASCADE,
    CONSTRAINT fk_libro_autor_autor FOREIGN KEY (autor_id) REFERENCES autores (id) ON
DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

init.sql – tabla prestamos

```
CREATE TABLE IF NOT EXISTS prestamos (
    id                        BIGINT      NOT NULL AUTO_INCREMENT,
    usuario_id                BIGINT      NOT NULL,
    libro_id                  BIGINT      NOT NULL,
    fecha_prestamo            DATE        NOT NULL,
    fecha_devolucion_prevista DATE        NOT NULL,
    fecha_devolucion          DATE        NULL,
    estado                    VARCHAR(20) NOT NULL DEFAULT 'ACTIVO',
```

```
    PRIMARY KEY (id),
    CONSTRAINT fk_prestamos_usuario FOREIGN KEY (usuario_id) REFERENCES usuarios (id),
    CONSTRAINT fk_prestamos_libro   FOREIGN KEY (libro_id)   REFERENCES libros (id),
    CONSTRAINT ck_prestamos_estado  CHECK (estado IN ('ACTIVO', 'DEVUELTO',
'RETRASADO'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

7. Docker

La aplicación se distribuye en dos contenedores orquestados con Docker Compose: **db** (MariaDB) y **app** (Spring Boot). El `Dockerfile` usa una construcción **multi-etapa**: la primera etapa compila con el JDK y Maven Wrapper y la segunda solo copia el JAR sobre una imagen JRE, más ligera.

Dockerfile

```
# ---------- Fase 1: compilación ----------
FROM eclipse-temurin:25-jdk AS build

WORKDIR /app

# Cachea las dependencias antes de copiar el código
COPY mvnw pom.xml ./
COPY .mvn .mvn
RUN chmod +x mvnw && ./mvnw -q dependency:go-offline

# Copia el código y compila
COPY src ./src
RUN ./mvnw -q package -DskipTests


# ---------- Fase 2: ejecución ----------
FROM eclipse-temurin:25-jre

WORKDIR /app

COPY --from=build /app/target/app-backend-p1-0.0.1-SNAPSHOT.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

En `docker-compose.yml` el servicio `db` monta `db/init.sql` y un volumen para persistir los datos, y dispone de un *healthcheck*; el servicio `app` espera a que la base de datos esté sana (`depends_on` con `condition: service_healthy`) y recibe la URL de conexión `jdbc:mariadb://db:3306/biblioteca` por variables de entorno. El puerto 8080 se publica en el anfitrión.

docker-compose.yml

```
services:
  db:
    image: mariadb:11
    container_name: biblioteca-db
    environment:
      MARIADB_ROOT_PASSWORD: root
      MARIADB_DATABASE: biblioteca
      MARIADB_USER: biblioteca
      MARIADB_PASSWORD: biblioteca
    volumes:
      - db_data:/var/lib/mysql
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    ports:
```

```
      - "3306:3306"
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 5s
      timeout: 5s
      retries: 20
      start_period: 10s

  app:
    build: .
    container_name: biblioteca-app
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:mariadb://db:3306/biblioteca
      SPRING_DATASOURCE_USERNAME: biblioteca
      SPRING_DATASOURCE_PASSWORD: biblioteca
    ports:
      - "8080:8080"
    restart: unless-stopped

volumes:
  db_data:
```

7.1. Puesta en marcha

Comandos

```
# construir las imágenes y arrancar app + base de datos
docker compose up --build -d

# comprobar el estado de los contenedores
docker compose ps

# abrir la aplicación
http://localhost:8080

# parar (y borrar también los datos con -v)
docker compose down        # docker compose down -v
```

8. Trabajo realizado

1. **Análisis y diseño**: definición de requisitos, modelo E/R en notación Chen con la relación N:M Libro–Autor y paso a modelo relacional.
2. **Mockups** de las pantallas principales para fijar navegación, tablas, formularios y botones antes de programar.
3. **Base de datos**: script `init.sql` con esquema, restricciones e inserción de datos de ejemplo.
4. **Modelo y repositorios**: entidades JPA con sus relaciones y validaciones; repositorios Spring Data con consultas derivadas.
5. **Controladores y vistas**: CRUD completo de libros y usuarios; listado y alta de autores; listado, alta y devolución de préstamos; página de inicio con estadísticas; layout común con Bootstrap 5.
6. **Reglas de negocio**: control de ejemplares disponibles al prestar/devolver y bloqueo del borrado de registros con préstamos asociados.

7. **Pruebas**: 12 tests automáticos con JUnit 5 sobre H2 en modo MariaDB (repositorios con `@DataJpaTest` y controladores con MockMvc).
8. **Diseño de la interfaz**: hoja de estilos propia con el estilo de ficha catalográfica, sellos de estado, signatura topográfica, accesibilidad WCAG 2.1 AA y adaptación a móvil.
9. **Contenerización**: Dockerfile multi-etapa y `docker-compose.yml` con MariaDB, volumen y healthcheck.
10. **Documentación**: esta memoria, diagramas, mockups y vídeo demostrativo.

8.1. Estado final por entidad

Entidad	Funcionalidad implementada
Libros	CRUD completo: listado, alta, edición y borrado; selección múltiple de autores; ISBN único validado; borrado bloqueado si tiene préstamos.
Usuarios	CRUD completo: listado, alta, edición y borrado; email único validado; borrado bloqueado si tiene préstamos.
Préstamos	Listado, alta (descuenta un ejemplar; no permite prestar sin ejemplares) y devolución con el botón «Marcar devuelto» (repone el ejemplar). El estado RETRASADO se calcula al consultar.
Autores	Listado (con número de libros de cada autor) y alta.
Inicio	Contadores de libros, usuarios, autores y préstamos activos (indicando cuántos van con retraso) y tabla de los 5 últimos préstamos.

8.2. Pruebas automatizadas

`./mvnw test` ejecuta 12 pruebas sobre una base de datos H2 en memoria (perfil `test`, modo de compatibilidad MariaDB):

Clase	Pruebas
LibroRepositoryTest (3)	Guardar un libro con dos autores y recuperarlo con ellos (N:M); existsByIsbn; findAllByOrderByTituloAsc.
PrestamoRepositoryTest (4)	Crear usuario, libro y préstamo y contar por estado; existsByLibroId; existsByUsuarioId; findTop5 de los últimos préstamos.
BibliotecaControllerTest (4)	Las páginas principales responden 200; crear un libro válido redirige y lo guarda; crear un libro sin título muestra errores; eliminar un libro con préstamos muestra un error y no lo borra.
AppBackendP1ApplicationTests (1)	El contexto de Spring arranca.

8.3. Capturas de la aplicación en funcionamiento

Capturas de la aplicación real ejecutándose con `docker compose up` en http://localhost:8080. La interfaz final mantiene la estructura de los mockups (barra oscura, listados con «+ Nuevo» azul, Editar amarillo, Eliminar rojo, devolución en verde) con el diseño de «ficha catalográfica» descrito en el apartado 5.5. Diferencias respecto a

los mockups: el botón de devolución se rotula «Marcar devuelto», los autores de un libro se marcan con casillas en lugar de una lista con Ctrl+clic, el panel de inicio permite devolver directamente y muestra el inventario de la colección, y los libros agotados no se pueden elegir al crear un préstamo.

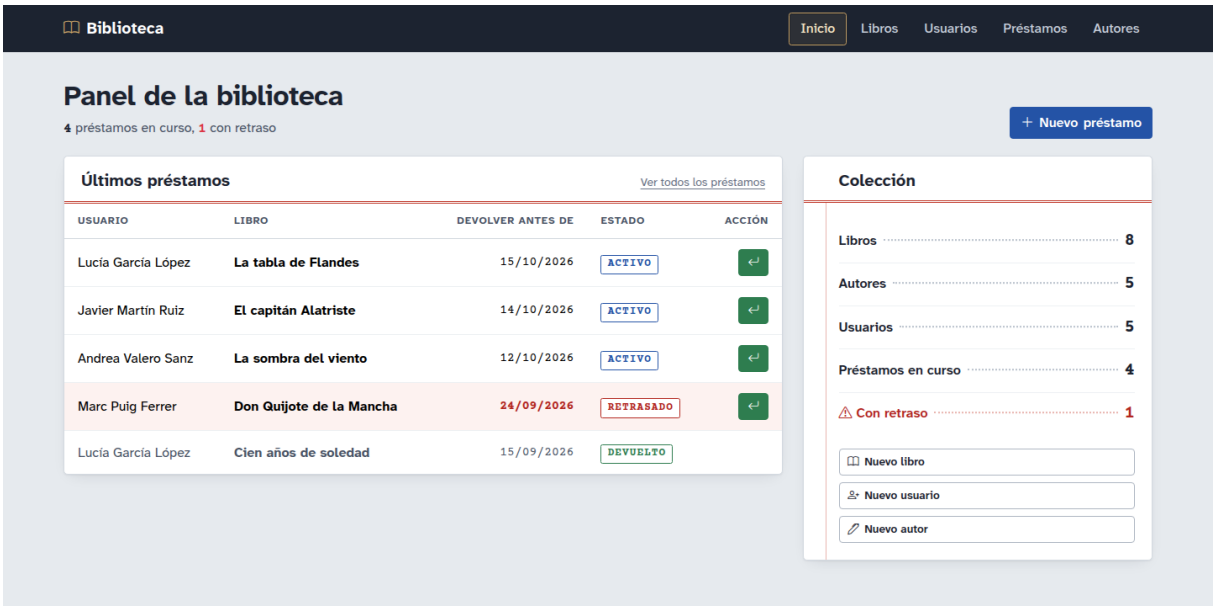


Figura 11. Captura – Página de inicio con contadores y últimos préstamos

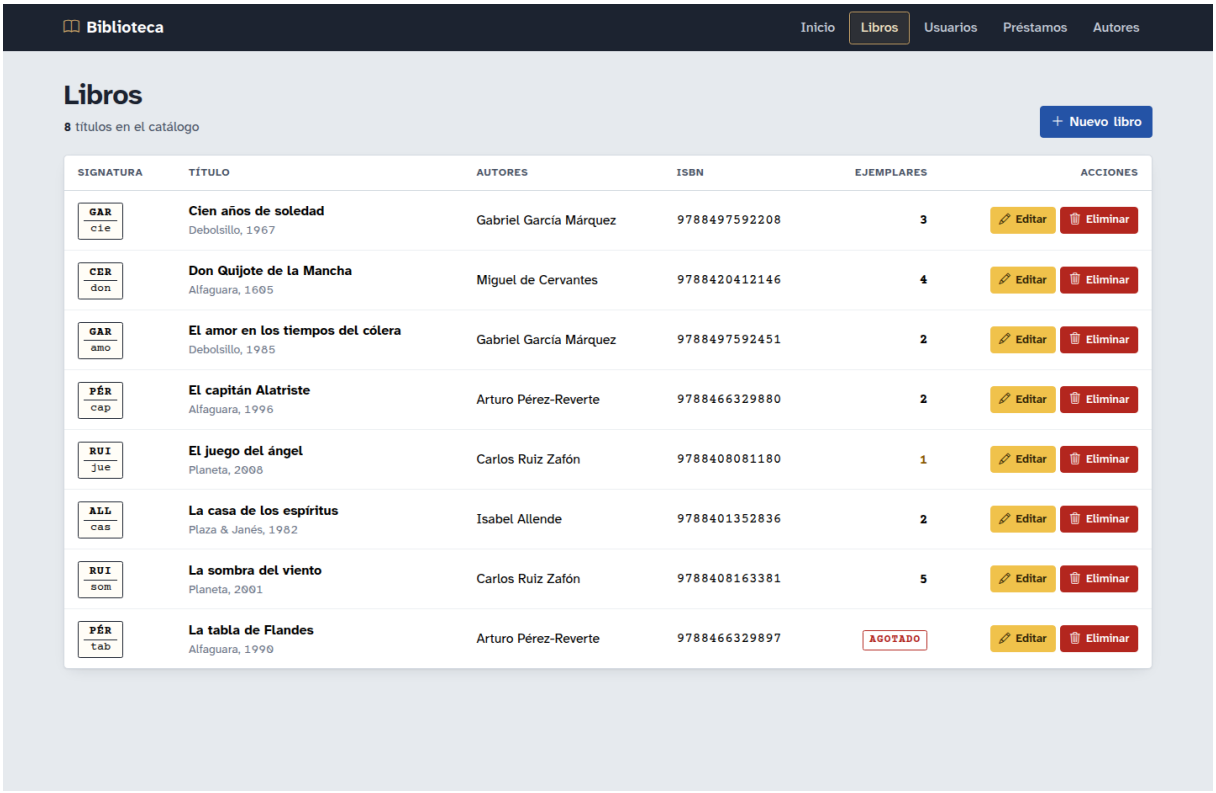


Figura 12. Captura – Listado de libros con sus autores

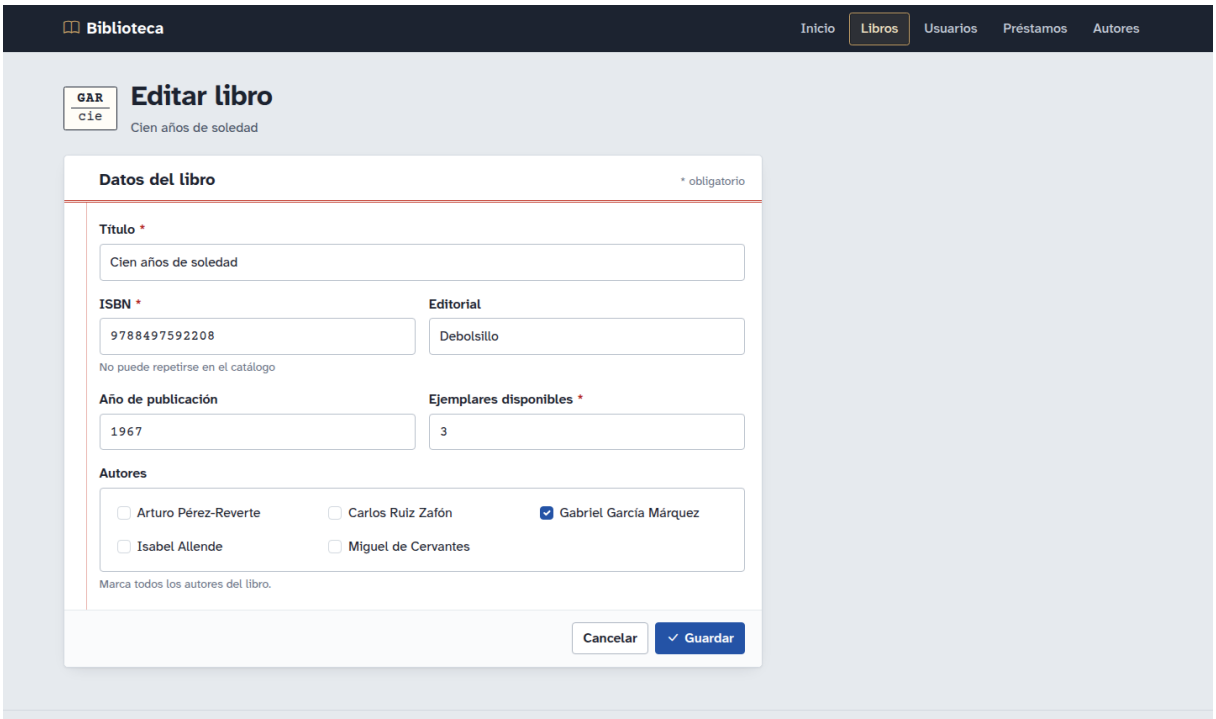


Figura 13. Captura – Formulario de libro con selección múltiple de autores

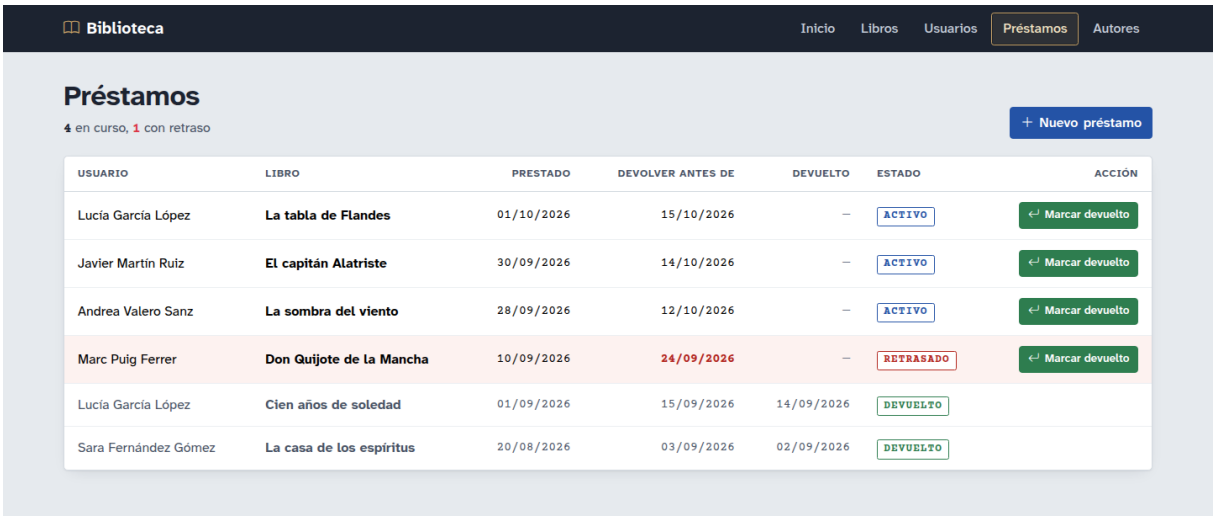


Figura 14. Captura – Listado de préstamos con estados y botón «Marcar devuelto»

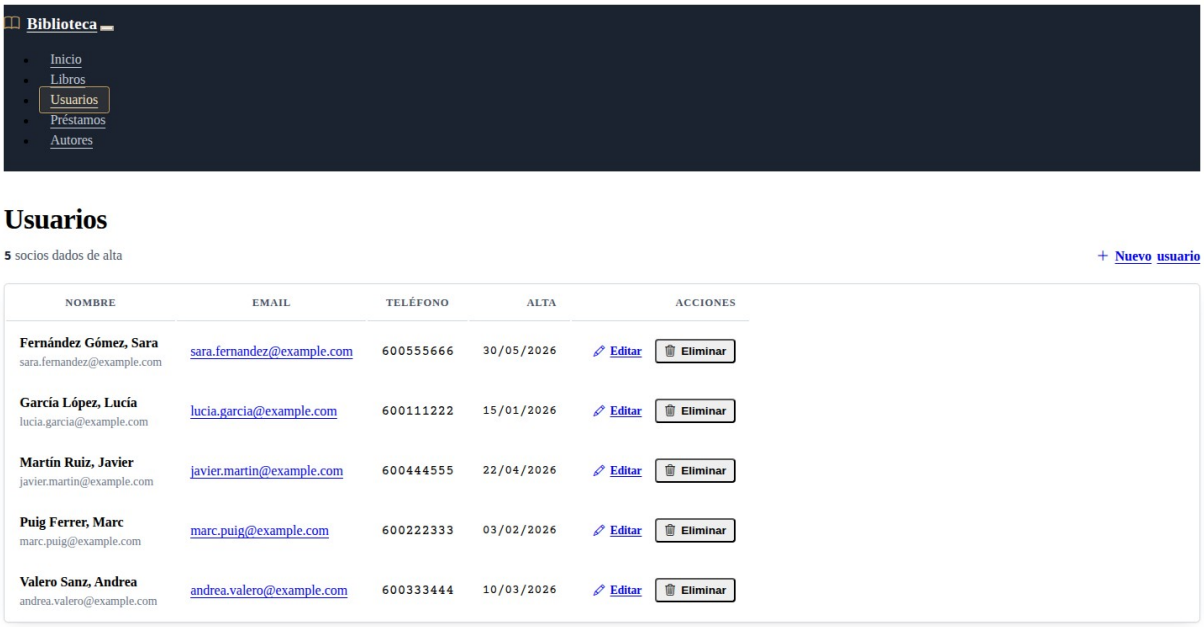


Figura 15. Captura – Listado de usuarios



Figura 16. Captura – docker compose ps (app y db en marcha) y códigos HTTP 200 de todas las rutas

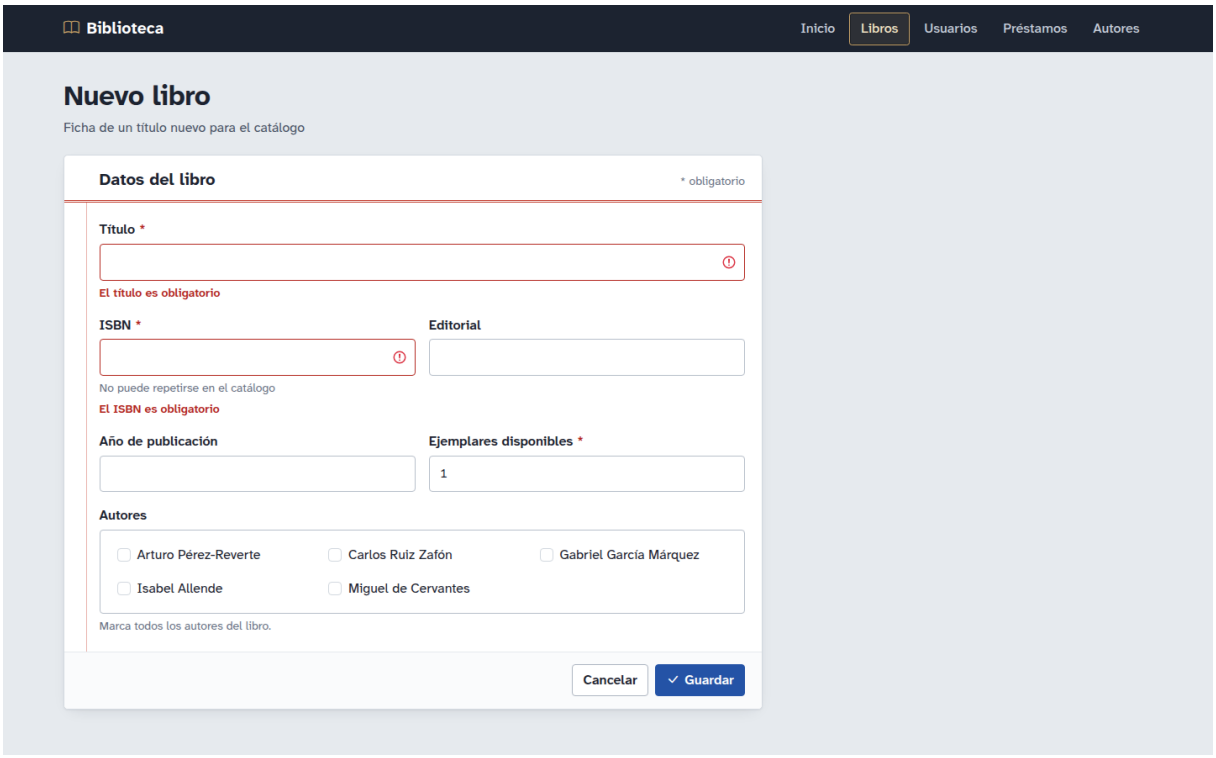


Figura 17. Captura – Errores de validación al guardar un libro sin título ni ISBN

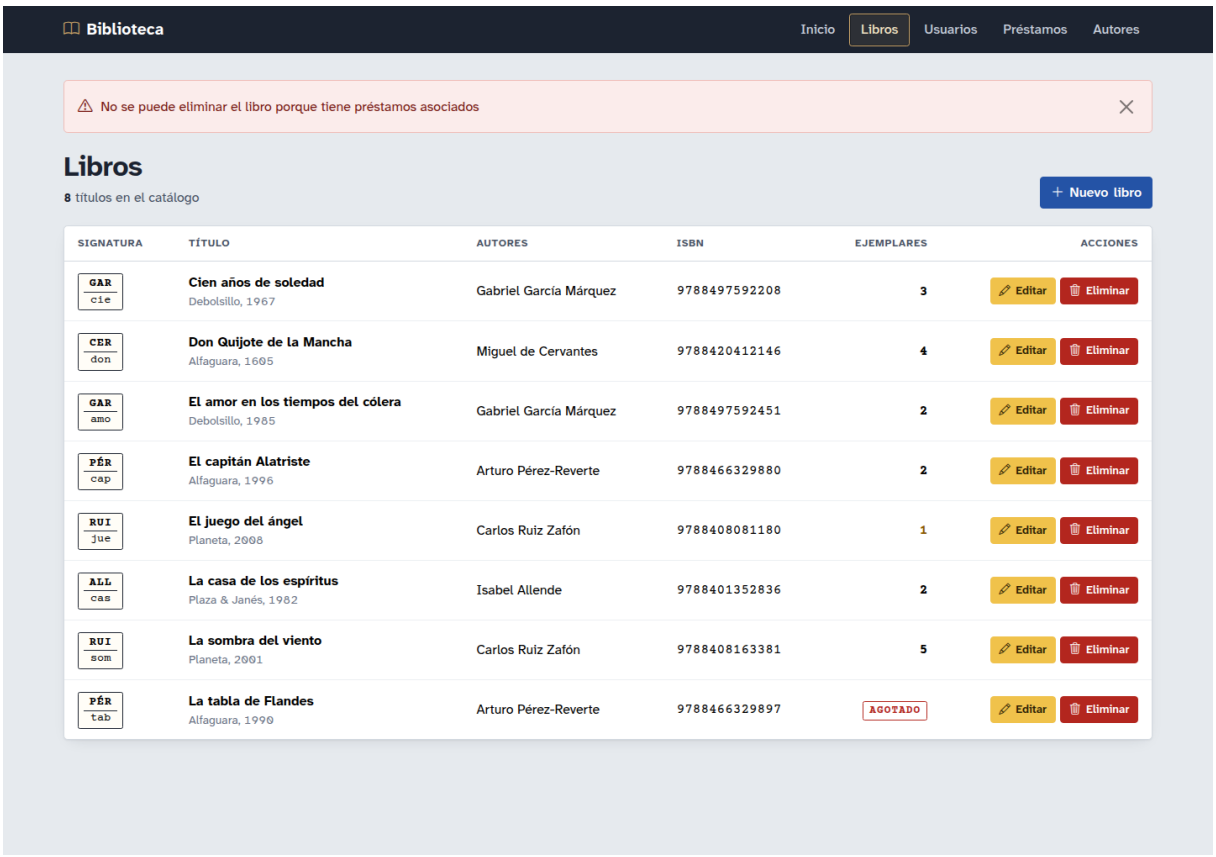


Figura 18. Captura – Borrado bloqueado de un libro que tiene préstamos asociados

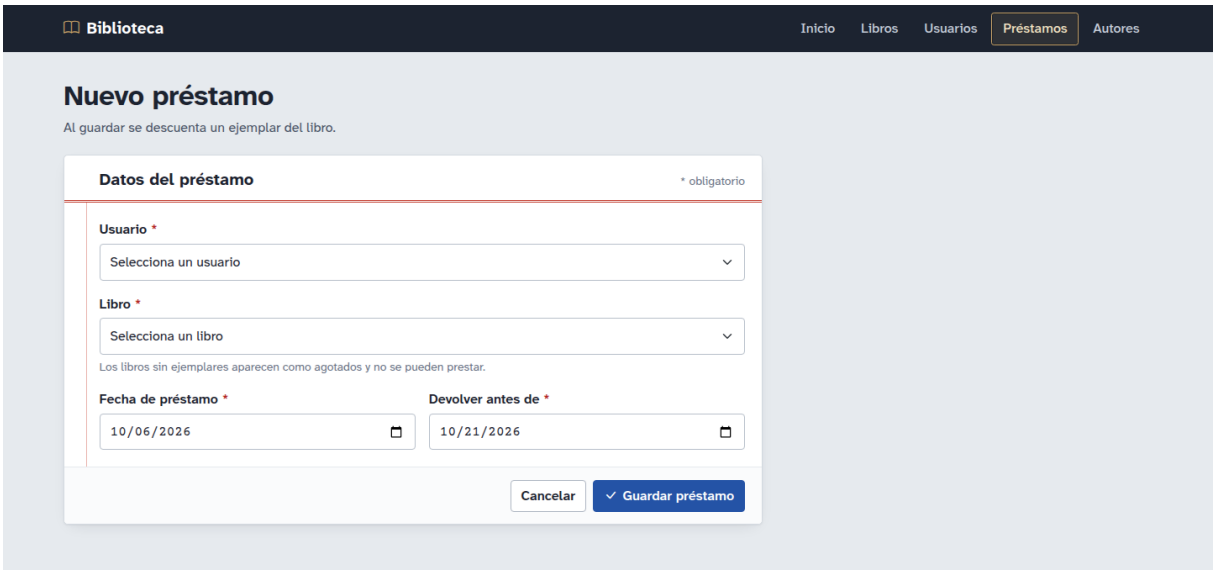


Figura 19. Captura – Formulario de nuevo préstamo

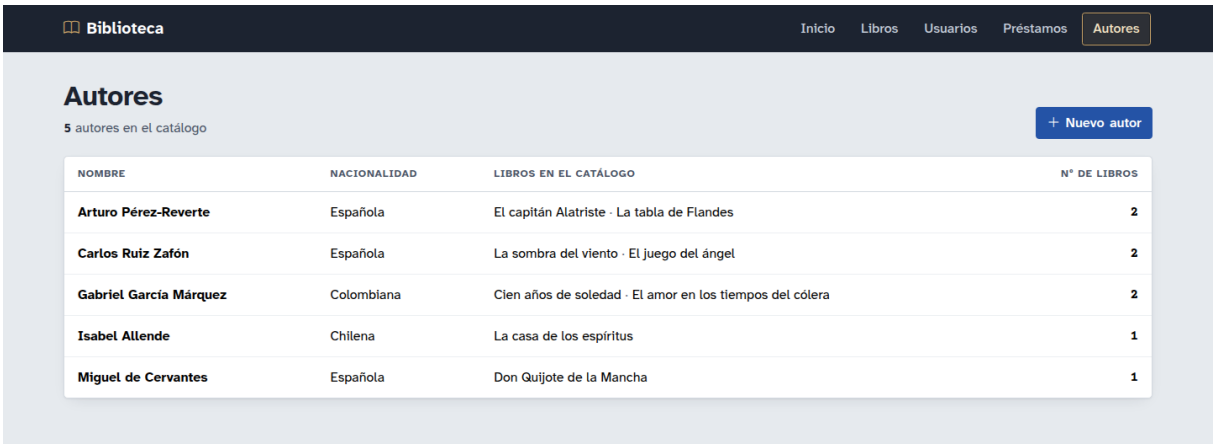


Figura 20. Captura – Listado de autores con el número de libros de cada uno

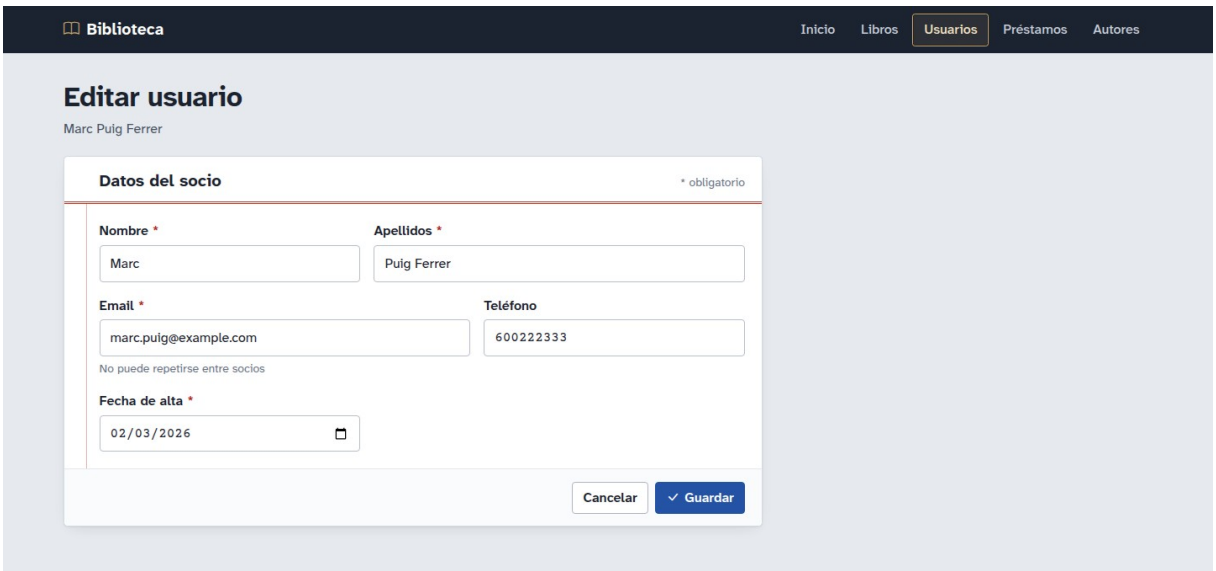


Figura 21. Captura – Edición de un usuario

8.4. Vídeo demostrativo

> ⚠️ **PENDIENTE · Enlace al vídeo de YouTube: https://youtu.be/_____ (añadir en VIDEO_URL del script o directamente aquí)**

Guion del vídeo: arranque con `docker compose up`, recorrido por el menú, alta/edición/borrado de un libro asignando varios autores, alta de un usuario, creación de un préstamo y su devolución, y comprobación de los datos en MariaDB.

9. Dificultades encontradas

- **Mapeo del enum de estado**: Hibernate, con MariaDB, genera por defecto una columna `ENUM` nativa, que no coincidía con la columna `VARCHAR(20)` + `CHECK` del script SQL y hacía fallar la validación del esquema (`ddl-auto=validate`). Se resolvió forzando el tipo con `@JdbcTypeCode(SqlTypes.VARCHAR)`.
- **Relación N:M en formularios**: enlazar la selección múltiple de autores con un `Set<Autor>` requiere que Spring convierta los identificadores recibidos en entidades; se resolvió recibiendo los ids con `@RequestParam List<Long> autorIds` y cargándolos con `autorRepository.findAllById`.
- **Orden de arranque en Docker**: la aplicación arrancaba antes de que MariaDB aceptara conexiones. Se añadió un *healthcheck* al servicio `db` y `depends_on: condition: service_healthy` en `app`.
- **Codificación de caracteres**: para que tildes y eñes se guarden y muestren bien se usó `utf8mb4` en la base de datos y `SET NAMES utf8mb4` en el script de inicialización.
- **Integridad al borrar**: eliminar un libro o usuario con préstamos provocaba un error de clave foránea; se comprueba antes con `existsByLibroId`/`existsByUsuarioId` y se muestra un mensaje en lugar del error.

10. Puntos no realizados

- **Autenticación y roles** (bibliotecario/socio): no se ha implementado Spring Security; cualquier persona con acceso a la URL puede gestionar los datos.
- **Tarea programada para RETRASADO**: el estado se recalcula al consultar los préstamos, no mediante una tarea `@Scheduled` ni con avisos al usuario.
- **Edición y borrado de autores**: de los autores solo se implementó el listado y el alta.
- **Edición y borrado de préstamos**: un préstamo se crea y se devuelve, pero no se puede modificar ni eliminar desde la interfaz.
- **Búsqueda, filtros y paginación** en los listados.
- **Pruebas contra MariaDB real**: los 12 tests usan H2 en memoria; no hay pruebas de integración con MariaDB ni pruebas de la interfaz en navegador.

11. Propuestas de mejora

- Añadir **Spring Security** con inicio de sesión y roles.
- Tarea `@Scheduled` diaria que marque como RETRASADO los préstamos vencidos y envíe un aviso por email al usuario.
- **Paginación y búsqueda** con `Pageable` de Spring Data (por título, ISBN, autor o usuario).
- **API REST** (`@RestController`) para un futuro cliente móvil o SPA.
- **Migraciones** de esquema con Flyway en lugar del script `init.sql`.
- Completar el CRUD de **autores** (editar y eliminar) y permitir corregir o anular préstamos.
- Pruebas de integración con **Testcontainers** (MariaDB real) además de los tests actuales sobre H2.
- Gestión de **ejemplares** individuales (código de barras por copia) y reservas.

12. Autoevaluación

Valoración según los cinco criterios de la rúbrica oficial (5 puntos cada uno). Para cada uno se indica lo que exige el nivel máximo y la evidencia de este proyecto que lo cumple:

Criterio (nivel máximo exigido)	Evidencia en el proyecto	Puntos
1. Diagrama E/R – al menos 3 entidades con todos sus atributos (mín. 2 por entidad) y relaciones, siendo una N a M.	4 entidades (Usuario, Libro, Autor, Préstamo) con todos sus atributos: 6, 6, 3 y 5 respectivamente, PK subrayada. Relaciones Usuario 1:N Préstamo, Libro 1:N Préstamo y **Libro N:M Autor**, con cardinalidades (apartado 3.2). Se añade el modelo relacional con la tabla intermedia libro_autor (3.3).	5 / 5
2. Mockup de la aplicación – al menos 2 interfaces detalladas que presenten al menos las 3 entidades, más el menú y los botones de navegación.	8 mockups detallados (apartado 4): inicio, listado y formulario de libros, listado y formulario de préstamos, listado y formulario de usuarios y listado de autores. Aparecen las 4 entidades, la barra de navegación (Inicio, Libros, Usuarios, Préstamos, Autores) y los botones Nuevo, Editar, Eliminar, Devolver, Guardar y Cancelar.	5 / 5
3. Modelos mediante Spring Data – al menos 3 modelos y sus relaciones.	4 entidades JPA (Usuario, Libro, Autor, Prestamo) y el enum EstadoPrestamo. Relaciones @ManyToMany/@JoinTable (Libro–Autor), @OneToMany/@ManyToOne (Usuario–Préstamo y Libro–Préstamo) y 4 repositorios JpaRepository con consultas derivadas. Los tests de repositorio verifican la relación N:M (apartados 5.1, 5.2 y 8.2).	5 / 5
4. CRUD mediante Spring – 1 CRUD completo con controladores y vistas, más listados del resto de modelos.	**Dos** CRUD completos (Libros y Usuarios: listar, crear, editar y eliminar) con controlador, servicio y vistas Thymeleaf, validación y mensajes. Del resto de modelos hay listados y algo más: Préstamos (listar,	5 / 5

	crear y devolver) y Autores (listar y crear). Panel de inicio con contadores (apartados 5.3, 8.1 y 8.3).	
5. Modificaciones del contenedor Docker – nuevas librerías y binarios para que el proyecto se ejecute y funcione correctamente.	Librerías nuevas en pom.xml (Spring Data JPA, Thymeleaf, Validation, driver mariadb-java-client) que el Dockerfile descarga en la etapa de compilación (dependency:go-offline). Dockerfile multi-etapa: JDK 25 para compilar y JRE 25 para ejecutar. docker-compose.yml añade el servidor MariaDB 11 con init.sql, volumen, healthcheck y variables de entorno. Se ha comprobado que funciona: contenedores en marcha y todas las rutas responden HTTP 200 (apartado 7 y figura 16).	5 / 5
Total		**25 / 25**

Se cumple el nivel máximo de los cinco criterios, en varios casos por encima del mínimo: 4 entidades en lugar de 3, 8 mockups en lugar de 2 y dos CRUD completos en lugar de uno. Lo que no se ha hecho (apartado 10: edición y borrado de autores y préstamos, autenticación, paginación) va más allá de lo que pide la rúbrica, así que no resta puntuación.

13. Webgrafía

- Spring Boot – Reference Documentation. https://docs.spring.io/spring-boot/index.html
- Spring Framework – Web MVC. https://docs.spring.io/spring-framework/reference/web/webmvc.html
- Spring Data JPA – Reference Documentation. https://docs.spring.io/spring-data/jpa/reference/
- Spring Guides – Accessing Data with JPA. https://spring.io/guides/gs/accessing-data-jpa
- Spring Guides – Validating Form Input. https://spring.io/guides/gs/validating-form-input
- Hibernate ORM – User Guide. https://docs.jboss.org/hibernate/orm/current/userguide/html_single/Hibernate_User_Guide.html
- Jakarta Bean Validation. https://beanvalidation.org/
- Thymeleaf – Tutorial: Thymeleaf + Spring. https://www.thymeleaf.org/doc/tutorials/3.1/thymeleafspring.html
- MariaDB – Documentación (Knowledge Base). https://mariadb.com/kb/en/documentation/
- MariaDB Connector/J. https://mariadb.com/kb/en/about-mariadb-connector-j/
- Imagen oficial de MariaDB en Docker Hub. https://hub.docker.com/_/mariadb
- Docker – Documentación. https://docs.docker.com/

- Docker Compose – Referencia del fichero. https://docs.docker.com/reference/compose-file/
- Bootstrap 5 – Documentación. https://getbootstrap.com/docs/5.3/getting-started/introduction/
- Bootstrap Icons. https://icons.getbootstrap.com/
- W3C – Pautas de Accesibilidad para el Contenido Web (WCAG) 2.1. https://www.w3.org/TR/WCAG21/